TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

8. Stack tecnologico

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026). Cada decision tiene su ADR en *documento 11 (Decisiones de arquitectura (ADR))* con las alternativas que se compararon, y responde a un driver de *documento 6 (Drivers de arquitectura)*.

Vista general

Capa	Tecnologia	ADR
App movil	Expo SDK 57, React Native 0.86, expo-router, NativeWind (Tailwind)	ADR-001, ADR-007
Interfaz	Template Luna adaptado (tema claro y oscuro)	ADR-007
Voz (entrada)	Grabacion con `expo-audio` en el celular; transcripcion con Whisper en el servidor (whisper.cpp o faster-whisper, API compatible con OpenAI); entonacion por texto y voz	ADR-014
Voz (salida)	VoiceStudio local (IndexTTS 2, emocion por vector), Azure Speech y OpenAI de respaldo; `expo-speech` como ultimo recurso	ADR-015, ADR-016
Backend	Node.js 22 y Express 4: monolito modular en arquitectura hexagonal, API REST `/api/v1` con OpenAPI	ADR-002, ADR-011, ADR-013
Base de datos	SurrealDB 3.x en la nube (SurrealDB Cloud), detras de un repositorio; SQLite solo en pruebas	ADR-012
Traduccion e IA	Qwen local (Ollama), Claude Sonnet de respaldo y motor de reglas espanol/ingles	ADR-004
Autenticacion	JWT de acceso (15 min) y token de refresco rotativo; contrasenas con bcrypt	ADR-006
Proteccion de datos	Mensajes cifrados con AES-256-GCM, consentimiento, eliminar cuenta, logs sin datos personales	ADR-010
Pruebas y CI	`node:test` (cobertura minima 80 %, hoy cercana al 96 %), pruebas contra SurrealDB, TypeScript, dependency-cruiser para las capas; GitHub Actions	ADR-013

Por que cada tecnologia y que alternativas se evaluaron

Capa	Elegida	Por que	Alternativas evaluadas
App movil	Expo (React Native)	Una sola base de codigo para iOS, Android y web; aprovecha el template Luna; se prueba en el celular con Expo Go.	Flutter (otro lenguaje, Dart, y no usa el template); nativo Swift y Kotlin (dos desarrollos para dos personas); web responsiva (microfono y voz limitados).
Backend	Node.js con Express, en arquitectura hexagonal	Mismo lenguaje que la app; monolito modular suficiente para el alcance; la traduccion es por turno, asi que REST basta. Los puertos y adaptadores dan limites claros sin el costo de los microservicios.	Microservicios (complejidad sin necesidad); capas clasicas sin puertos; WebSocket o streaming (solo para traduccion simultanea); Python FastAPI (otro lenguaje); Java Spring Boot (mas pesado).

Capa	Elegida	Por que	Alternativas evaluadas
Base de datos	SurrealDB en la nube	El dominio es una cadena de relaciones que guarda como enlaces nativos y recorre sin JOIN; el esquema con ASSERT y UNIQUE deja las reglas del dominio en la base; es multimodelo (reportes flexibles y, a futuro, busqueda vectorial de modismos sin otra base); gestionada en la nube, con usuarios de permisos limitados.	SQLite (archivo local, no sirve para varios usuarios ni produccion); PostgreSQL (muy maduro, relaciones con JOIN); SDK oficial de SurrealDB (dependencia extra).
Traduccion	Qwen local, Claude de respaldo y reglas	Adapta modismos y tono por region y explica la adaptacion; Qwen corre en el servidor (sin costo por mensaje y sin sacar datos); si falla, sigue funcionando.	DeepL o Google Translate (literal, no explican modismos); solo Claude (costo y datos a un tercero); solo reglas (no escala); NLLB o Argos (no siguen instrucciones de tono).
Voz (entrada)	`expo-audio` + Whisper en el servidor	Expo Go no trae reconocimiento de voz nativo; grabar en el celular y transcribir en el servidor funciona igual en iOS, Android y web, y deja medir el volumen para estimar la emocion. Whisper corre local, sin costo por mensaje.	Dictado del teclado (no mide la voz ni sirve en Expo Go de forma uniforme); reconocimiento nativo (requiere un build propio de Expo); servicios de nube de pago.
Voz (salida)	VoiceStudio local con IndexTTS 2; Azure y OpenAI de respaldo	Voz neuronal con emocion graduada (enojo, alegria, tristeza) en espanol e ingles, local y sin costo por mensaje.	`expo-speech` (suena robotica, sin emocion real); solo Azure (clave y costo); solo OpenAI (costo y datos a un tercero).
Autenticacion	JWT corto y refresco rotativo	Recomendado para apps moviles; permite cerrar sesion de verdad (revocacion).	Sesiones en servidor (guardan estado); Firebase Auth o Auth0 (dependen de un tercero y sacan datos personales).
Datos personales	Cifrado de mensajes y logs sin PII	Cumple la Ley 1581 de 2012 sin dependencias nativas.	Cifrar toda la base (dependencia nativa; se deja al proveedor en produccion).
Nombres del dominio	Espanol en el codigo (la base en ingles)	Trazabilidad con el modelo de dominio y las reglas de negocio.	Todo en ingles (rompe la trazabilidad).

Estructura del codigo (capas)

El backend sigue una arquitectura hexagonal (puertos y adaptadores, ADR-013):

- `backend/src/dominio`: reglas puras (validaciones, errores de negocio, analisis de idioma, tono y region, prosodia de la voz y expresion de cada tono).
- `backend/src/aplicacion`: un caso de uso por epica (cuentas, sesiones, conversaciones, traduccion, voz, catalogo, reportes).
- `backend/src/puertos`: contratos `Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `Notificador`, `Cifrador`, `Hasheador` y `FirmadorTokens`.
- `backend/src/adaptadores`: HTTP (Express) de entrada; de salida SurrealDB y SQLite, Qwen, Claude y reglas, Whisper, VoiceStudio, Azure y OpenAI, correo y cifrado.

- `backend/src/composicion.js`: elige los adaptadores segun el entorno y arma los casos de uso.
- `backend/db/surreal/schema.surql`: esquema de la base. `backend/openapi.yaml`: contrato de la API.
- `frontend/app`: pantallas (expo-router); `frontend/components/talklate`: componentes de TalkLate; `frontend/lib`: cliente de la API, reglas, grabacion y voz.
- `componentes/template/native-luna/`: template original, sin cambios.
- `backend/.dependency-cruiser.cjs`: reglas de dependencia entre capas (`npm run arch:check`).

Como correrlo

- Backend: `backend/` con `npm start` en `http://localhost:3000` (configuracion en `backend/.env`, ver `backend/.env.example`).
- App: `frontend/` con `npm run web` o Expo Go (SDK 57).
- Motores locales de IA (Ollama + Qwen, Whisper y VoiceStudio) y medicion: `backend/VOICESTUDIO_MAC.md`.